Coordinated Robotic Manipulation - UR3 and UR3e

Context

In this practice, two robotic arms, a **UR3 and a UR3e**, work together to complete the same task. Each robot has a different job, and they communicate so they know when it is their turn to move.

The **UR3e picks up a block**, moves it to another position and places it there. Once this is done, it sends a message to the UR3.

The **UR3 then rotates the block by 180 degrees**, changing its orientation. After finishing, it sends a message back to confirm that the task is complete.

The robots communicate using a **TCP socket connection**, which allows them to send simple messages to each other and coordinate their movements.

The main goal of the practice is to understand how two robots can **communicate, synchronize their actions and work together safely** in the same robotic task.



Figure 1. Setup of the practice, with the UR3e on the left and the UR3 on the right.

UR3 Side - Server

The UR3 robot script functions as the **server** in a collaborative task controlled via a Python proxy. The main function, `ur3_collaboration()`, organizes a coordinated interaction between two robotic arms through a communication protocol and a well-defined sequence of motions.

The process begins with the UR3 moving to a predefined **home** position, achieved using a joint-space configuration with moderate acceleration and velocity parameters to ensure smooth and reliable movement. This guarantees a known and safe starting disposition.

Next, it attempts to **open** in the UR3 a TCP **socket** connection to a predefined IP address, the one of the PC host, on port 8080. If it fails to load, an error message is displayed on the teach pendant and the execution halts to prevent undefined behavior.

Once the link is successfully established, the UR3 **waits** specifically for the string `"client ready"`. This message confirms that the UR3e has also turned on properly and is **ready** for this one to proceed. If it is not received within a set timeout, it displays an error and terminates.

Upon obtaining it, the UR3 moves to its **"operate"** position, defined as a specific Cartesian pose where the collective operation is going to take place.

The UR3 then **notifies** the UR3e that it has **moved** by sending a message with the string `"server moved"`.

Following this, the UR3 **waits** for the message with the string `"block placed"` from the UR3e, which indicates that it has completed its **pick-and-place** operation and the **block** is now positioned.

The UR3 then reads its **current Tool Center Point (TCP)** pose, modifies the orientation by adding π radians to the **Z-axis**, effectively **rotating** the **tool 180°**, and performs a flip operation. It then returns to its original arrangement using another command, completing a full turn.

After finishing, the UR3 **notifies** the UR3e sending a final message with the string `"operated"` to signal **operation** completion.

The UR3 returns to its **home** position, clearing the workspace.

The UR3 then **closes** the **socket**, ensuring a clean shutdown.

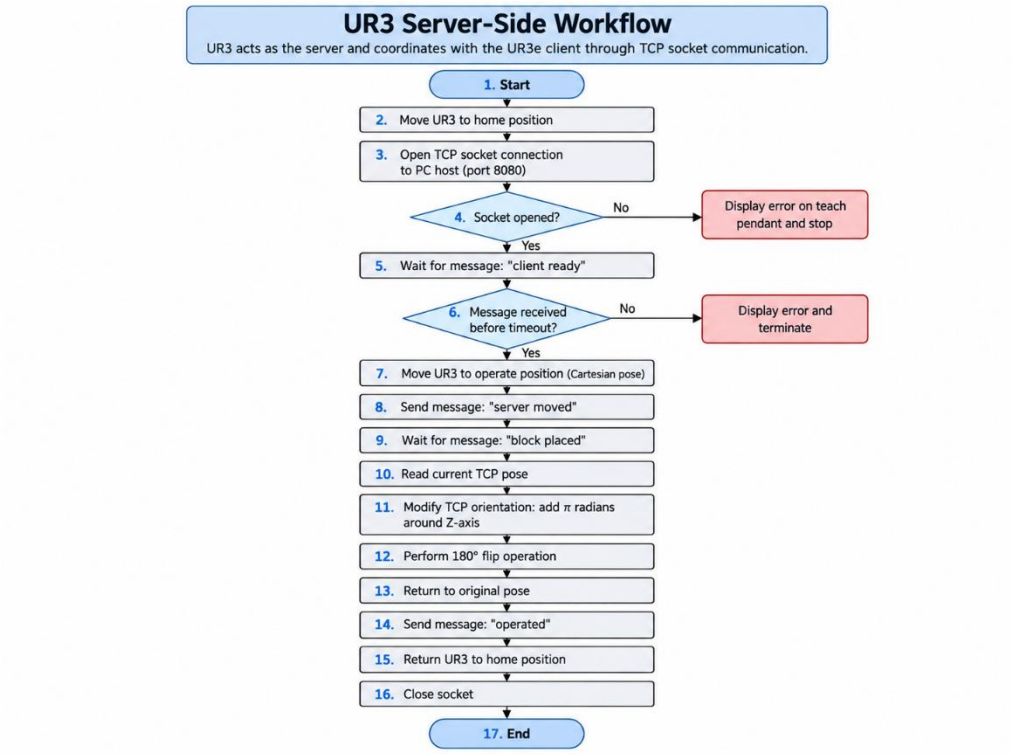


Figure 2. UR3 server-side workflow.

UR3e Side - Client

The UR3e robot script serves as the **client** in this collaborative task. The function `ur3e_collaboration()` complements the previous one.

It starts with the UR3e moving to its own **home** position, different from the other, but equally designed to establish a predictable and sheltered placement.

The UR3e then **opens** a TCP **socket** connection via port 8080. If this fails, an error message is shown and it halts immediately.

Prior to simultaneity, the UR3e computes both a **pick** position and an **approach** position 10 cm above, then moves linearly to the approach position to ensure a secure descent.

Once in position, the UR3e sends a message with the string `"client ready"` to **notify** the UR3 that it is **ready** and that it can proceed.

The UR3e then **waits** for the `"server moved"` string message, ensuring the UR3 has completed its **move**.

After this, the UR3e completes the **pick** operation. It goes down to the pick position, closes the gripper, makes a short delay to stabilize the grasp and simulate a realistic work, and retires to the approach position.

The UR3e then carries the **block** to a predefined **place** position, using a similar two-step approach with a difference height of 4 cm. It releases the piece opening the gripper, pauses briefly, and retracts again to the highest point of the two.

Upon successful set-up, it **notifies** that the **block** is already **placed** to the UR3e sending the message with the string `"block placed"`, which triggers the next action.

The UR3e then **waits** for the `"operated"` string message, which confirms that the UR3 has completed its **operation**.

At this point, the UR3e returns to its **home** position.

The UR3e **closes** the **socket**, finalizing the teamwork.

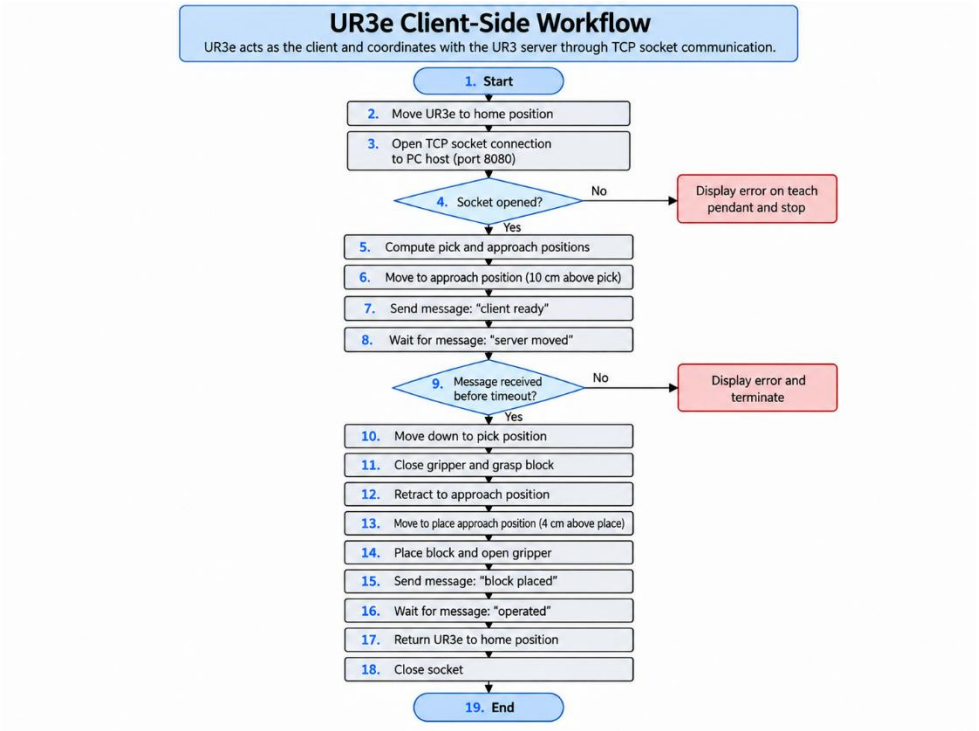


Figure 3. UR3e client-side workflow.

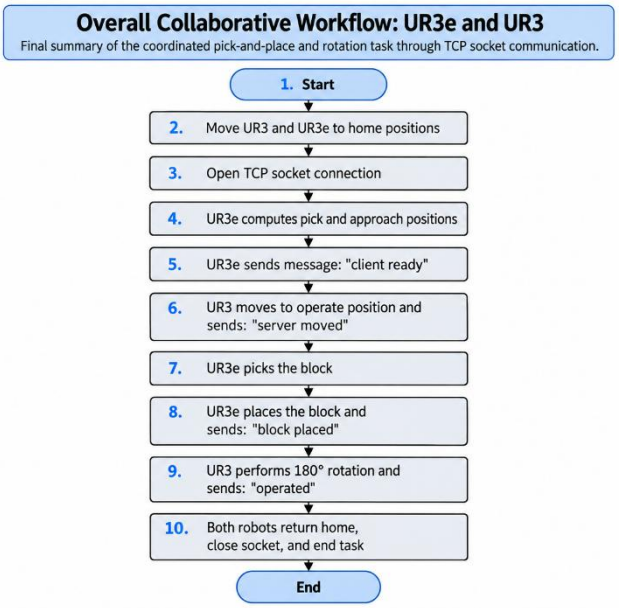


Figure 4. UR3e client-side workflow.

Conclusion of the project:

This practice showed how two robotic arms can work together by combining **robot motion, communication and synchronization**. The UR3e was responsible for moving the object, while the UR3 performed the rotation, so both robots had to exchange messages and wait for the correct moment before continuing with the task.

One of the main challenges was making the **socket communication** reliable, especially when messages were delayed, received in the wrong order or a timeout occurred. Another important difficulty was synchronizing both robots correctly so that their movements did not interfere with each other.

We also had to carefully adjust the **joint configurations, Cartesian poses and approach positions** for both the pick-and-place and rotation movements. This required several tests and corrections to obtain repeatable movements and avoid collisions.

Overall, the practice helped us understand that in a collaborative robotic system, it is not enough to program the movements of each robot separately. The **communication between the robots, the order of the actions and the accuracy of the programmed positions** are all essential for the complete system to work safely and correctly.